

Shulker — Complete Technical Guide

From security fundamentals to the full implementation · Hackathon study companion

What this document covers

Part 1 — Theory	APIs, authentication vs authorization, OWASP API Top 10, the vulnerability classes Shulker targets, redaction, rate limiting.
Part 2 — Build	Every technology used and <i>why</i> : Node zero-dep backend, TypeScript, SQLite, SSE, React, Gemini.
Part 3 — The scanner	Engine phases, SafeHttpClient chokepoint, how each detector actually works, AI + deterministic verification.
Part 4 — Product	Dashboard, scan wizard, templates, external-target mode, CLI, reports.
Part 5 — Quality	The 60-check test gate, bugs we hit and fixed, verification philosophy.

6

workspaces

4+1

vuln classes +
controls

60

e2e checks

0

backend deps

3

consent gates

The one-sentence pitch

Shulker scans web APIs for authorization flaws the way attackers actually exploit them — it takes a valid low-privilege credential and **proves** with reproducible HTTP evidence whether it can read another user's data, reach admin functions, or extract hidden fields — using AI only for hypotheses and deterministic code for verdicts.

Part 1 · Security theory you must know

1.1 What an API is, and why it is the new attack surface

An API (Application Programming Interface) is a contract that lets machines talk: clients send HTTP requests like `GET /users/42` and receive structured JSON. Modern apps are hybrids of many APIs — mobile apps, SPAs, partner integrations, microservices. Unlike a website, an API has **no UI to hide behind**: every capability the frontend uses is directly callable by anyone with the URL and a credential. The frontend is not a security boundary; the server is.

1.2 Authentication ≠ Authorization (the core distinction)

	Authentication (AuthN)	Authorization (AuthZ)
Question	Who are you?	May you do this?
Mechanism	Password, API key, JWT, OAuth token	Ownership checks, role checks, ACLs

Failure class	Credential theft, token leakage	BOLA, BFLA — the classes Shulker detects
---------------	---------------------------------	---

The breach pattern that powers this whole project: attackers rarely break cryptography. They *authenticate successfully* as a normal user, then exploit missing authorization — reading object IDs that belong to someone else, or calling admin functions the UI never showed them. Authentication says "this is Alice"; authorization was supposed to say "Alice may only see Alice's invoices" — and often nobody implemented that second check server-side.

1.3 The vulnerability classes Shulker detects

Class	OWASP	Definition	Concrete attack
BOLA — Broken Object-Level Authorization	API1:2023	The server does not verify that the requested object belongs to the caller.	<code>GET /vehicles/101</code> → own car, 200. <code>GET /vehicles/102</code> → <i>someone else's car, also 200</i> . Attackers enumerate IDs and scrape every user's data (aka IDOR).
BFLA — Broken Function-Level Authorization	API5:2023	Privileged functions are reachable by non-privileged callers.	<code>PATCH /users/42/role {"role":"admin"}</code> sent as a normal user — the admin panel was hidden in the UI, but the server never checked the role.
Excessive Data Exposure	API3:2023	Responses return more fields than the client needs — full DB rows including sensitive ones.	<code>GET /users/42</code> returns <code>passwordHash</code> , internal flags. The mobile app "never shows" them — attackers read raw HTTP.
Weak Rate Limiting	API4:2023	No throttling on sensitive operations.	Unlimited login attempts → credential stuffing; unlimited ID probes → BOLA at industrial scale; OTP brute-force.
Security Controls (positive)	—	Checks that <i>pass</i> : the server does enforce the rule.	Admin endpoint returns 403 for regular users → recorded as PASS, giving defenders an attested baseline.

1.4 Supporting theory used in the build

- **JWT (JSON Web Token)** — signed token of form `header.payload.signature`. Carries identity claims; the scanner extracts it at login and replays it as `Authorization: Bearer ...`. Its recognizable shape (`eyJ...`) is also what the redaction engine hunts for.
- **API keys** — static per-client secrets in a header (`x-api-key`, `x-goog-api-key`). Simpler than JWT but long-lived — leakage is high-impact, hence redaction and header-based injection in Shulker.
- **OpenAPI 3.0.3** — machine-readable API contract: `paths`, operations, parameters, `components.securitySchemes`. Shulker's parser derives endpoints *and* expected response schemas from it (the schema is the baseline for exposure detection).
- **Redaction** — replacing secrets with `<REDACTED>` before persistence. A scanner stores evidence; if it stored raw tokens it would itself become a breach vector.
- **Rate limiting** — server-side throttling (429 Too Many Requests). Detected politely: Shulker sends a bounded probe burst and observes.
- **IDOR** — Insecure Direct Object Reference, the classic pre-2023 name for BOLA.

Part 2 · The stack — every technology and why

2.1 Monorepo layout (npm workspaces)

```
shulker/
├─ apps/
│   ├─ sandbox/      ShulkerLab – deliberately vulnerable demo API (port 8700)
│   ├─ scan-api/     Backend: REST + SSE, scan orchestration, reports (port 8600)
│   ├─ web/          React + Vite operator dashboard (port 5173)
│   └─ cli/          `shulker scan` – same engine, CI gate with --fail-on
├─ packages/
│   ├─ scanner-core/ Scan engine, SafeHttpClient, detectors, AI module, parser
│   └─ shared/       Types, severity ordering, redaction engine
├─ scripts/         dev.sh · e2e.mjs (60 checks) · verify-demo.mjs
└─ data/            SQLite DB + generated reports
```

Why a monorepo? One repo, one `npm install`, one `npm run dev` — critical for a hackathon demo. Workspaces give each package its own `package.json` while sharing a single lockfile and `node_modules`, and shared types flow between engine, server and UI with zero publish steps.

2.2 Technology choices and their reasons

Layer	Choice	Why (the honest engineering reason)
Language	TypeScript, everywhere	One language across engine, server, sandbox, UI. Shared types mean an engine change that breaks the server fails at <i>compile time</i> , not during the demo.
Runtime	Node ≥ 20.10 with <code>--experimental-strip-types</code>	Runs TS directly — no build step for the backend . Edit and restart. Also enables the zero-dependency decision below.
HTTP server	Raw <code>node:http</code> + a hand-rolled route table	Total control over SSE streaming, CORS and error shape. Express would add a dependency without adding capability we need.
Database	<code>node:sqlite</code> (built into Node)	Persistent scans/findings/evidence with zero setup — no Postgres container to run on judge Wi-Fi. Single-file DB in <code>data/</code> .
Zero-dep backend	No npm runtime packages at all	A security tool must not blindly trust its supply chain. Every line that touches target HTTP, crypto or storage is our own auditable code.
Live updates	Server-Sent Events (SSE)	Scan progress is one-way server→client. SSE gives auto-reconnect and event replay over plain HTTP — simpler than WebSockets and observable with curl.
Frontend	React 19 + Vite, hand-written CSS	Fast HMR while iterating; ~76 kB gzipped bundle; a custom dark "SOC console" design system with CSS variables instead of a component library.
AI	Google Gemini (REST)	Hypothesis generation only. Deliberately <i>not</i> in the verdict path — see §3.5.
Testing	<code>node:test</code> + custom e2e script	The e2e boots the sandbox in-process, runs a real scan, and asserts 60 checks including redaction sweeps and the authorization matrix.

2.3 Why this stack wins a hackathon demo

- `npm install && npm run dev` → everything up in ~10 seconds, no services.
- No API keys required to demo (AI is optional; deterministic rules run without Gemini).
- Works fully offline except AI hypotheses and external-target scans.

2.4 The safety architecture (what makes it a responsible scanner)

Guard	Mechanism
Bounded requests	Engine counts every request; <code>budget</code> (10–400) and <code>rateLimitProbeCount</code> (5–25) are hard caps. A scanner that can DoS its target is itself a vulnerability.
Timeouts	Every fetch wrapped in <code>AbortController</code> (2–30 s).
Response caps	512 KB streaming truncation — no unbounded memory.
No redirects	<code>redirect: 'manual'</code> — evidence reflects the real first response.
Redaction	JWTs, password fields, token-shaped strings → <code><REDACTED></code> before DB/evidence/reports/SSE.
Target gate	<code>validateTarget(url, allowExternal)</code> — loopback/private always OK; public only with external mode.
Consent gates	(1) Settings toggle, (2) per-scan authorization checkbox, (3) server-side validation of both.
Header hygiene	User-supplied headers cannot override <code>Host</code> / <code>Content-Length</code> (injection blocked + tested).

Part 3 · Inside the scanner

3.1 Scan lifecycle (engine phases)

```
POST /api/scan → validateTarget + confirmAuthorized + buildAuthHeaders ↳ ScanEngine.run(): parse
Parse OpenAPI/Postman/HAR → endpoints + auth + schemas authenticate Login actors (Alice/Bob) OR
static credentials (API key/bearer) hypotheses Resource/ownership discovery → AI hypotheses
(optional) testing BOLA · BFLA · Exposure · Rate-limit detectors (bounded) findings Correlate,
dedupe, assign severity + reason done Persist result, emit SSE SCAN_DONE, reports on demand
```

3.2 SafeHttpClient — the single HTTP chokepoint

Every request the scanner makes passes through one class that enforces timeout, size cap, no-redirect and redaction of anything stored. `defaultHeaders` injects user static credentials once so **every** detector inherits the auth context automatically. This chokepoint is why budget and safety guarantees are enforceable at all — there is no side door around it.

3.3 How each detector works (the real logic)

Detector	Procedure	Verdict rule
BOLA	1. Discover list endpoints and object ids per actor. 2. Record which objects belong to Alice. 3. Replay Alice's read/update requests using Bob's token on Alice's object ids.	200/OK with the <i>other</i> actor's token → confirmed , with both responses as evidence and a curl PoC.
BFLA	Identify privileged operations (admin paths, role-mutating writes). Call them as a regular user, and with tampered role claims.	Success status as non-admin → confirmed (CRITICAL if role escalation). 403 → recorded as passing control.
Exposure	Diff actual response JSON fields against the OpenAPI schema for the operation; scan for sensitive patterns (<code>passwordHash</code> , tokens, PII names).	Sensitive undocumented field present → confirmed HIGH with the offending payload as evidence.
Rate limit	Send ≤ 25 rapid, polite probes at one sensitive endpoint (e.g. login). Observe statuses, headers (<code>RateLimit-*</code>), latency trends.	No 429 / no backoff signal across the burst → potential MEDIUM (deliberately conservative).

3.4 Severity: deterministic, explainable

Findings carry `severity` + `severityReason` computed by rules (type × confirmability × sensitivity of leaked field × privilege gained), never by the LLM. CI users can gate with `shulker scan --fail-on high` and trust exit codes.

3.5 AI + deterministic verification (the flagship idea)

Why the AI never has the final say

LLMs hallucinate; a vulnerability scanner that reports hallucinated findings is worse than useless. Shulker's division of labor: **Gemini** reads the spec and ranks *which* endpoints/objects look juicy (hypotheses, prioritization); **code** then tries each hypothesis with real HTTP and either produces evidence or drops it. Result: zero hallucinated findings, by construction. Without a Gemini key the scanner still runs fully on deterministic rules.

3.6 Contract discovery & templates

- **Imports** — OpenAPI (JSON/YAML), Postman Collections, HAR files, manual entry, spec-by-URL — all parsed into one endpoint model with auth classification.
- **Quick API Templates** — one click for Gemini / OpenAI / GitHub / Stripe generates a valid OpenAPI 3.0.3 spec in memory, auto-fills target + auth (e.g. Gemini → `x-goog-api-key`), and shows a view/copy/download preview. No file wrangling in the demo.

Part 4 · The product surface

4.1 Dashboard pages

Page	What it does
Overview	KPIs with deltas vs previous scan, scan status timeline, recent findings, severity donut, top vulnerability types, activity feed.
New Scan	Scan mode (ShulkerLab sandbox / Real API) → Quick Templates → target + external toggle + warning → contract source → discovery preview (protected vs public counts) → auth (None / API Key / Bearer / Basic + custom headers) → profile (budget, probes, AI) → authorization → Run. Live SSE monitor with phase timeline.
Findings	Per finding: severity, title, affected endpoint, expected vs actual behavior, redacted evidence, remediation, copy-PoC.
Evidence	Raw (redacted) request/response exchanges behind every finding.
Reports	HTML / Markdown / JSON generation and download.
Discovery & Intelligence	API Inventory (merged sources), Attack Surface graph, Coverage report, Resource Graph, Activity feed.
Settings	Target security (local-only, confirmation, redaction, Allow External Targets), scanner defaults, AI status, history clear.

4.2 One-click demo flow (what judges see)

New Scan → Local Sandbox → Run Demo Scan ~30s later: 5 CONFIRMED – CRITICAL BFLA role escalation · 3× HIGH BOLA (vehicles/service-records/invoices across users) · HIGH exposure (passwordHash) + 1 POTENTIAL (no rate limiting on /auth/login). Every token redacted.

4.3 External authorized mode

- Toggle **Allow External Targets** + per-scan authorization checkbox → public HTTPS targets and public spec URLs accepted.
- Static auth: API key (custom header name), Bearer, Basic — injected into every request, redacted everywhere.
- Live-verified against the public Swagger Petstore: 19 endpoints discovered, 8 tested, API key never stored in plaintext.
- Templates make the flow file-less: pick provider → key from provider console → scan.

4.4 CLI (same engine, CI gate)

```
shulker scan \  
  --target http://127.0.0.1:8700 \  
  --openapi http://127.0.0.1:8700/openapi.json \  
  --api-key YOUR_KEY \  
  --fail-on high      # exit 1 if any high/critical finding → blocks the pipeline
```

Terminal and dashboard report **identical findings** from the same engine run — the CLI is the DevOps story: scan in CI, fail PRs that introduce BOLA.

4.5 REST API (backend surface)

Route	Purpose
POST /api/scan	Start scan — target, spec/useSandboxSpec, auth{}, budget, allowExternal, confirmAuthorized
POST /api/demo	One-click local ShulkerLab scan
GET /api/scans/:id · /events	Result JSON · SSE live stream (with replay)
POST /api/reports	Generate html/md/json report
POST /api/inventory/import import-url manual	Contract imports → inventory
GET /api/graph · /coverage · /activity	Attack-surface graph, coverage, activity
GET/POST /api/settings	Defaults + external-targets toggle

Part 5 · Quality, bugs & verification story

5.1 The test gate (run after every change)

```
1. npx tsc --noEmit (web · scan-api · scanner-core) 2. npx vite build 3. node scripts/e2e.mjs 60
checks: [1] full sandbox scan (endpoints, tests, BOLA/BFLA/exposure confirmed) [2] redaction sweeps
(no raw JWT / password hashes anywhere) [3] reports (markdown + html, contain PoC, leak-free) [4]
authorization matrix (external refuse/allow, Host-injection blocked, api-key/bearer/basic/custom
header building) [5] template matrix (4 templates: valid 3.0.3, validator-pass, auth-marked) 4. node
scripts/verify-demo.mjs live HTTP against a freshly booted server
```

5.2 Real bugs we hit and fixed (the engineering story)

Bug	Root cause	Fix & lesson
Giant warning triangle filled the viewport	Inline SVGs had no intrinsic size; inside flex containers they stretched to container size.	Global CSS cap (15–16 px) on icons inside notices/panels/toasts. <i>Layout bugs can be one missing CSS property — root-cause, don't patch symptoms.</i>
tsc: "(',, expected)" cascade	Unquoted hyphenated object keys (<code>arrow-left</code>) are invalid identifiers; a malformed SVG path compounded it.	Quote keys; rewrite icon. <i>Dump exact file bytes (JSON.stringify) when grep output looks fine.</i>
Wizard stepper never advanced	Compared against scan <i>phases</i> (only exist while running) instead of wizard steps.	Separate STEPS state for the wizard; phases live only in the monitor. <i>Two state machines must not share one index.</i>
JSX compile errors in auth form	<code><div></code> closed as <code></label></code> ; JSON braces inside a JSX attribute.	Unified tags; <code>placeholder={'{"openapi": ...}'}</code> expression. <i>Attributes take JS expressions, not raw JSON.</i>
Servers died between terminal commands	Execution sandbox reaped background processes between commands.	Made verification single-command and self-contained (in-process sandbox). <i>Design tests to not depend on ambient state.</i>
<code>spec:"sandbox"</code> rejected	Scan route expects spec text or <code>useSandboxSpec:true</code> ; a URL is not a spec.	Fixed usage; documented that spec-URLs go through import-url. <i>Validate the contract between layers — read the route source.</i>
Engine failed on spec URL	Engine deliberately accepts text only.	Fetch-then-parse; Petstore scanned successfully live. <i>Single-responsibility boundaries show up as integration friction.</i>
Old branding after rename	Stale process still bound to :8600.	Killed listener; re-verified health. <i>Check WHICH process answered, not that something answered.</i>
Workspace links broken after rename	<code>@sentinel/*</code> → <code>@shulker/*</code> invalidated lockfile links.	<code>npm install</code> + full gate re-run. <i>Renames are not text edits — re-link and re-verify.</i>

5.3 Verification philosophy

- **Trust nothing ambient** — every check boots what it needs and asserts on real output.

- **Security claims need sweeps** — "redaction works" is a regex sweep over the entire serialized scan result, not a hope.
- **Matrix-test the gates** — authorization logic is exactly where you test both the allow *and* the refuse path.
- **Green means re-runnable** — the whole gate is two commands anyone can run before judging.

Cheat sheet: 30-second answers for judges

Q: Why not just ask an LLM to find vulns? Hallucination — verdicts need proof; AI only prioritizes.

Q: Is the scanner dangerous? No — budgets, timeouts, redaction, three consent gates, local-by-default.

Q: Why zero dependencies? Supply-chain trust; everything is auditable; SQLite is built into Node.

Q: Does it work without AI? Yes — deterministic rules are the default; Gemini is optional acceleration.

Q: Real APIs? Yes — external mode with static auth, live-verified on Swagger Petstore.